

CI-0118 Lenguaje Ensamblador

Laboratorio 4

Fecha de entrega: Viernes 22 de mayo, 23:59

En este laboratorio, usted desarrollará una pequeña biblioteca de procesamiento de imágenes en escala de grises. El programa principal estará escrito en C++, pero la parte más costosa del procesamiento será implementada en lenguaje ensamblador x86-64 usando instrucciones SIMD AVX2.

El objetivo es comparar una implementación tradicional en C++ con una implementación optimizada en ensamblador. Además, deberá utilizar macros de NASM para organizar el código SIMD.

La operación que se aplicará a la imagen será una combinación de:

1. Aumento de brillo con saturación.
2. Umbralización.

Para cada píxel de la imagen:

Se le aumentará el brillo al píxel, sumándole al valor del píxel el valor del brillo. Luego la umbralización corresponde a cambiar los píxeles cuyos valores son mayores al umbral a blanco (valor 255) y los que son menores a negro (valor 0). Es decir, la imagen resultante será una imagen en blanco y negro.

Una imagen en escala de grises puede representarse como una matriz de píxeles, donde cada píxel es un valor entero entre 0 y 255.

Por ejemplo:

0 representa negro
255 representa blanco
128 representa gris medio

En memoria, la imagen se almacenará como un arreglo unidimensional:

```
unsigned char* data;
```

Si la imagen tiene ancho width y alto height, entonces el píxel ubicado en la fila y y columna x se encuentra en: `data[y * width + x]`

En este laboratorio, el procesamiento se realizará directamente sobre ese arreglo.

En este laboratorio debe escribir dos funciones, una en alto nivel y otra en ensamblador que apliquen la transformación explicada a una imagen en escala de grises en formato pgm.

Las funciones deben recibir:

- Imagen de entrada.
- Imagen de salida.
- Cantidad total de píxeles.
- Valor de brillo.
- Valor de umbral.

Para el código ensamblador debe utilizar macros para organizar su función.

En mediación tiene disponible el código base en C++.

Para las instrucciones SIMD utilice los registros YMM y considere utilizar las siguientes instrucciones:

- `vmovd`
- `vpbroadcastb`
- `vmovdqu`
- `vpaddusb`
- `vpmaxub`
- `vpcmpeqb`
- `vzeroupper`

Puede usar otras si lo considera necesario.

Rúbrica:

La función de C++ procesa correctamente la imagen: 20%

La función de ensamblador procesa correctamente la imagen: 30%

Utiliza por lo menos dos macros en el código ensamblador: 20%

Usa correctamente las instrucciones SIMD: 30%